# Active InferAnt Stream 014.2 ~ Generalized Notation Notation for Generativ

*ActiveInferAntStream | Generalized Notation Notation for Generative Model Supply Chains | 1:57:07*

All right. Hello. Welcome, everyone. I'm going to start this stream with a GitHub push.

All right. It is June 8th, 2025, and this is Active Inference Stream 14.2. First, where we're going today in terms of AI art, to quickly give a little visual beginning.

No. I'll just show. Sorry about that. We're just working up to this last moment to get this going.

Where we're headed is using RxInfer, multi-agent trajectory planning example, and using GNN to give an exact reconfiguration.

So we're going to be using generalized notation notation to yield this simulation in RxInfer.

So there's a couple steps to go over. This is just sort of a live working presentations.

And to be refined and polished, I just wanted to get it all into this one stream, given all these recent updates.

So to the stream agenda and also people watching live, I will look forward to your comments and ideas.

Let's start with looking at the areas we'll go into.

First, just some overview on the tools. I'm using Cursor 1.0, which came out recently.

GNN, generalized notation notation. Check out stream 14.1 if you want to see more.

Using LLMs via Cursor, mainly Claude 4.

And it's going to be about reverse and forward engineering in the RxInfer setting.

Okay. Where we're headed is a robust information supply chain from plain text configuration to executable simulation.

So here's what that is going to look like again with the AI art.

We have GNN, the plain text GNN, moving through all of these stages, the triple play, graphical on through executed simulation.

So whether we think about that as meeting in the middle between the forward specification of GNN and all of these different kinds of things that we might want to do with GNN,

like the type checking, all the modules of the package.

And the golden spike moment is meeting in the middle with the reverse engineering of RxInfer examples and the forward engineering of GNN package.

So we can end up with this meeting in the middle between the plain text, which gives us all these benefits of plain text, reproducibility, etc., and all these functionalities of GNN.

In the main first section, I'll go over what that specifically was.

Talk about the reverse engineering in the RxInfer example, from script to more of a package,

and then meeting that package with a forward engineering pass from the GNN schema and pipeline on through the needed configuration.

And that's the golden spike.

Again, just a metaphor.

There have been many changes to the GNN package and still a few more that could occur with logging and so on,

but I'll go through a few sections of change and documentation.

A little bit more on the golden spike and the analogy.

Then we'll see what people comment where that goes.

And I have a little bit more on the other side.

And I have a little bit of manual writing as well in terms of thinking about what that means from an organizational perspective and for active inference and for the institute and all that.

Okay.

Going to the package.

All right.

All right.

In the doc RxInfer folder, the first thing is this short script clone RxInfer examples.

And this clones a repo, which is a fork of the main RxInfer examples that I have on my personal GitHub account.

And this, I sync it up with the main examples repo, which is awesome that they're putting out more and more and different.

And then in the support folder on my fork, first, there's a setup Julia script to get your environment ready.

And then there's a notebooks to scripts, Julia, which takes the notebooks from the examples and it turns them all into scripts just by extracting out the code blocks.

So they don't all necessarily work on the first pass, but it gets all the content.

Again, to this forward reverse engineering idea.

Here's the process.

So first, and the links to all this are in the video description.

First, use that notebooks to scripts script to convert the notebook into a script.

Sometimes it might run away.

Other times you might need to change certain things about the control flow.

And first step is to just lock in the single script.

So what that looks like in the case here.

In, so then it clones in.

This is work I did on the fork to get, this is the reverse engineering part with RxInfer.

Scripts, advanced examples, multi-agent trajectory, archive.

So this is the working single script version of the multi-agent trajectory model.

So just for context, where this takes us is these four agents that do a negotiation of avoidance collision and obstacle avoidance.

So different, different collision avoidance.

And that's happening with, at the heart of it all, this at model block, as always, in RxInfer, that is used by the path planner inference step.

So this is the, kind of the kernel.

And then RxInfer is making that kernel of the generative model, the explicit probabilistic joint distribution, makes it look a lot more like the math or the analytical description of the generative model.

And everything else is pushed into the inference methods.

So that's that kind of idea of writing shorter at model blocks that distinguish different generative models.

And then everything else is like rendering that out into the graphical model with the stereotyped inference routines.

From that single script, take it to more of a configurator modular approach.

So that was probably several hours, several dozen prompts, but not something that couldn't be done eventually in an automated fashion.

To split up that single script, that which was only lightly modified from the notebook format, which is 339 lines.

To modify it from single script into, in this case, almost 10 different scripts.

Some of them are really short, but they clarify.

And there's other ways that it probably could have been disassembled or disarticulated.

This separates out different functions and also adds a lot of visualization and logging methods.

Third step.

Third step.

From that multi-script package, all the hard-coded variables take it out to a config.toml file.

So this is like a little domain-specific language, just for this one example in this one case, where this gives all the configuration needed.

So instead of just having the values coded in, the scripts look to this central config standalone file.

There's multiple ways or stages of generalizing the variables, like the dimensional state spaces, initial parameterizations, constraint variables, number of agents, simulation variables like time steps, into this config block.

So that is going from, and all along the way, confirming that the simulation is still running.

So that was the reverse engineering of Artics and Fur.

So it's kind of like the original notebook was like a handmade chair.

And this was like reverse engineering it so that we could have this cartridge, which gets run like a disassembled IKEA factory, separating out the methods.

Whereas the original example is great and really informative in terms of a blog post.

So it's just readable, introduces things sequentially, and shows what RX and Fur is doing, and gets to those exact simulations.

This shows like one path through the flow, and it's a custom-made artifact.

And again, that reverse engineering process was taking it into these different disassembled components,

but exactly reproducing it, and then pulling out all the configuration into a config.

So that's the reverse engineering of the RX and Fur side.

Now in the forward rendering,

this is going from a GNN file to that config.

Because if we know from that reverse engineering phase that getting to that config is going to allow the execution of the simulation,

then the other side of that meeting in the middle is getting the GNN file into that config format.

So here is, in this examples folder,

a GNN style markdown file

with all of the parameters that need to be specified in the config.
So that when you're in SRC,
and you run main,
main is going to run all of the
all of the steps which are turned on
of the 14 steps,
which are each their own numbered script with the same name module.
Whichever ones you have enabled,
which is in
the new file in pipeline module config,
you can turn whichever of these
modules on or off.
That's different from last time.
And
that RX and Fur
GNN example
gets passed through
step one,
checks the files.
Step two,
does set up to the virtual environment.
Step three,
does any tests.
Step four,
type checker
output.
We can look in the type checking
and see,
for example,
that RX and Fur,
even though these
resource estimates
are not
currently specifically accurate,
especially compared across different packages,
but the
resources
are calculated

for that
RX and Fur
example
just by virtue of it being a GNN.
Then,
there's
let's continue looking through the
step five,
exports
into
a bunch of different outputs.
Six makes visualizations
and then of interest here is nine,
which renders
what we
want
into the Tomil.
And eleven,
it's going to call the LL.
So,
here's the exports.
This is the re-export
of the GNN
file
and the
export of the GNN file
in other
formats
like XML
and other
graph
formats.
That's step five.
Step six,
visualization.
We get
the
matrix

visualization
of
all of the
state spaces
and ontology
assertions
of
this
model.
So,
like,
this is
an interesting
example.
In this
engineering
RX and Fur
notation
namespace,
A is often
used for state
transitions,
whereas in the
active inference
textbook,
B is used for
state transitions.
But it's an example
of why we don't
want to have
a welding
between letters
and ontology
terms.
We want to have
the flexibility
to use the
same letters

for different
ontologies and
language in
different settings.
So,
it's a perfect
example of that,
of just the
ontology
assertion space
and all the
variables can be
described
just because it
was GNN
file.
Now,
we get to
step nine.
Oh,
it didn't run
completely.
To run it
again,
Python 3
main.
This will
run in
the render
folder
in SRC.
In the
RxInfer
section,
there's a
Toml
generator.
So,

this will
read in
the GNN
and then
output the
Toml,
which is
the one
that we
knew that
we wanted
from the
RxInfer
site.
Then,
this script
in the
RxInfer
folders
and doc
demonstrates
the GNN
RxInfer
pipeline for
multi-agent
trajectory
planning.
It performs
two-step
validation.
So,
first,
it runs
a baseline
execution
from the
clone
of the

RxInfer
examples.jl,
which doesn't
use GNN.
Then,
those
files
for the
example
are all
copied over
into this
folder right
here,
multi-agent
trajectory
planning,
and it's
logged
in the
running
of the
script
that the
config was
replaced
with one
regenerated
from the
GNN.
So,
it's a
clean
functional
replacement
experiment
to show
that the

GNN
structured
file
here
can be
used as
part of
this
bigger
pipeline,
which
includes
the
type
checking,
the
category
theory,
all these
different
kinds of
ontology
connections,
and yield
the target
TAML
that we
know
will make
the
simulation
work,
or the
reverse
engineered
disarticulated
RX
and FUR

example.
So,
I hope
that that
has been
conveyed.
It took
a long
time,
I think,
overall to
get here,
but it
had long
been the
triple play
vision,
that we
would have
a unified
plain text
format for
the math
diagrams and
visualizations and
executable code.
So,
it's like,
okay,
how do we
get triple
play integrity?
So,
we could have
things like,
okay,
well,
you change

the visual
representation
with a
drag and
drop,
and then it
changes the
executable
code.
It's like,
how is that
going to
work formally?
Well,
a lot of
the examples
scripts and
scientific papers
used like
an all-in-one
script,
which can
make reproducing
the example
simpler and
more monolithic,
but then give
fewer degrees of
freedom,
because you
might be just
like 700 lines
in the middle
of the script,
and it's not
clear how you
change certain
aspects more

structural about
the model.
people.
So,
how do we
get to
the ability
for there
to be a
triple play
where changes
in one can
be traced to
changes in
the other?
And that was
where Jakob
and I were
thinking,
maybe we have
some sort
of
intra
or infra
lingua
with a
generalized
notation
notation.
Let's find
that quote.
So,
RxInfer,
or any
other software
we could try
doing a
reverse

engineering of
a PyMDP
script,
gives the
ability to
set
everything
needed
except the
state space
of the
generative
model.
So,
there's still
a lot to
be explored
in terms
of how
do we
customize
control
flows
and more
simulation
environment
scale
factors.
And also,
a lot more
to say,
but suffice
to say
that for
simple
reproducing
of
standalone

examples,
this reverse
engineering
forensic phase
is a
very
important
proof of
concept.
inactive
inference,
research
models are
often conveyed
through
assemblages of
natural language,
pseudocode,
programming language,
analytical formulas,
and pictorial
representations.
In this paper,
we present
GNN as a
flexible and
expressive language
tailored for
expressing active
inference models
and encompassing
various relevant
aspects of
languages,
including
ontology,
morphology,
grammar,

and pragmatics.
By leveraging
GNN as an
active
inferlingua
or interlingua,
infralingua,
supralingua,
intralingua,
we aim to
bridge and
respect gaps
among different
modeling approaches
in order to
facilitate
interdisciplinary
research.
So,
how can this
be used?
Well,
for our
education,
we can
have these
well-tempered
generative model
infrastructural
pipelines
and specific
well-documented
interpretable
configurations
that help us
understand and
communicate and
figure out what

different generative
models and
process flow
elements are
doing.
There's also a
lot of
engineering uses.
So,
there could
and will be
many use cases
where GNN is
not a front
feature of
the product,
it's just a
useful,
increasingly
useful,
open source
and stack
for
connecting the
dots
between
existing
examples
and ways
to
modify
and design
with those
motifs
and then
increasingly
forward
use

of this
kind of
metaprogramming
program synthesis
type approach
to generative
modeling,
which is going
to have all
these benefits
for making
them easier
to design,
like just
being able
to prompt
an LLM
and say,
make me a
GNN file
that represents
this,
like we'll
do probably
later in
this stream,
all the way
on through
having the
resource
estimation,
all these
features that
we want
for any
generative
model can
be abstracted

and brought
into this
infrastructure-grade
pipeline so
that once
there are
more and
more paths
and understanding
about how
do you get
more and
more generically
or in all
these different
situations to
executable
simulations,
using this
pipeline and
putting in
the cartridge
helps separate
and give a
lot of
reproducibility
and expressivity
to the compute
that happens
in between.
So there's the
reverse engineering
approach,
which is just
taking scripts
exactly as
they are
and within

that language
or in more
of a port
or transfer,
doing an
exact reproduction
effort.
And then from
there, maybe
certain pieces
can be flipped
out, but it's
always good to
know that we
can exactly
reproduce using
the same
variable names,
the same
ontology
assertions,
so that becomes
kind of like a
substrate of,
well, paper
one modeled
attention this
way and in
natural language
they described
attention this
way.
Paper two,
in a different
programming language,
in a different
natural language,
said this about

attention,
and here's
how we can
use ontology
across the
natural language
and the
computational
elements to
talk about
where those
are similar
or different.
Buzzing can
be used for
different functions,
so for example,
once we have
the config and
the ability to
separate out
these different
variables, we
could see
what are the
perturbations that
are allowable,
because we
didn't change
any of the
logic or the
control flow.
So for example,
if the number
of agents were
simply changed
to 50,
then as it

is currently
written,
there might
need to be
some other
refactorizations
of state
spaces,
which just
changing NR
agents wouldn't
exactly do.
It might just
be part of a
loop and then
the loop would
break because
it's like,
wait, there
were only four
because we
only had four
initial locations.
So then you
go, okay,
okay, okay.
how could we
develop
configuration
spaces where
a number of
agents at
a programmatic
level had a
relationship to
instantiations of
matrices and
other features

like knowing
that you needed
to have as
many of these
blocks as
you do
agents.
So in this
exact situation,
again, it was
first just an
exact reverse
engineering, but
from that reverse
engineering,
through insight
and through
program
synthesis and
fuzzing
approaches, we
can explore
like what
happens when
the DT is
0.01 or
do a program
sweep across
generate
simulations where
DT ranges
from this to
this while the
number of
iterations ranges
from that to
that.
So then that

gives this
ability to
explore what
are the
functional and
the performance
aspects,
optimization.
visualization.
And then we
can go even
further, especially
as the
visualization and
the DiscoPy
features
improve.
So here, not
every single
variable was
captured by the
DiscoPy category
theory description,
but it wasn't
the focus here,
but that would be
something that could
be worked on.
And again, in
general, the
goal is a
wide range of
defined and
checked GNN
formats, ranging
from discrete
time, continuous
time, all these

different kinds of
scenarios.
We continue to
build out that
space and
neighborhoods of
different models
which can be
interpreted a
certain way.
So there's a lot
of things that
could happen with
the processing
pipeline, like how
do we know that
this PyMDP one,
we only want to
render into PyMDP
and the Rx
infer.
So okay, should we
include that
information in the
file and or in
the file name?
How should that be
handled by the
different modules?
Those are the kinds
of software
improvements that
would then allow
us to, when we
run it, right now,
let's just say we
had it perfectly
working for

rendering PyMDP to
PyMDP and Rx
infer to Rx
infer.
We still might get
50% errors and
then it would try
to chase around
fixing, but it's
didn't need to be
fixed.
But then if we
had some examples,
which are actually
quite few, if any,
where we have the
same exact analytical
situation being
cross-rendered into
PyMDP and Rx
infer.
So those would be
some interesting
ways to check that
all of the cross-
rendering is working
well.
things like that.
So, I'm going to

agenda.
Okay, returning to
the agenda, and if
anyone has any
suggestions or
questions, write them.
Otherwise, I'm just
going to try to get

through the rest of
the agenda, go to
some of the manual
writing, see if
there's some
interesting thing that
we could look, maybe
look at the
documentation and
just kind of have
this informal
bringing together of
all these recent
developments over the
last couple of
days, so that at
least it's been
worked through and
put out there for
those who want to
explore it, and
then it will return
in more forms over
the coming months.
So, the golden
spike was meeting in
the middle between
reverse engineering of
the n equals 1
rx infer example
into the
components into
the config form.
Pipeline execution
summary
is a JSON file.
So, what's cool
about this is

different steps can
be tucked in with
the arrow.
So, you can look at
just the logging for
each step, and it
probably makes it
more machine
readable as well.
But, like, here's
the full logging for
the step 5, and
then step 6,
visualization, and
so on.
In terms of how
the pipeline
overall has
changed, big
picture is still
the exact same,
which is all the
documentation is in
docs.
The rx infer script
is in there for
now, just to be
close to rx infer
cloned repo, and
all the source code
is in SRC.
So, you just delete
the output folder,
and then run
main.py
here.
It runs
whichever

of the
modules are
configured to run
in config.py.
It will run
those sequentially.
Each of them
have a
function, and
this is all
very
documented up
in the repo.
the
thing.
Okay.
GNN,
computational
science,
deployment
science,
multi-agent
systems,
being able to
design and
describe these
systems,
and it
starts quietly,
being able to
write and read
and modify
GNN files and
markdown that
starts to flow out
through more and
different places,
better and better.

that's just one
trace through an
information supply
chain, which
hops across
different operating
systems and
mines and
servers and all
of that.
However, with
this GNN
entry point or
meeting point,
there's the ability
to do some
verified computing
and tracing a
little bit better
from which
generative model
and which data
inputs and so
on are run
how,
when,
to yield
what outputs.
and that flow
from abstract
concept to
concrete execution
is unified
slash unifiable.
Some of the
most
important ways
that people could

explore from here,
getting the
GNN repo
examples working
on their own
machine,
making GitHub
issues or
comments with
ways to
improve it,
seeing what
kinds of
scenarios and
value propositions
they'd like to
do themselves,
in which case
just go for it
and report back,
measure back,
work,
or to partner
or support the
institute so
that we can
have these
kinds of
tools being
developed very
functionally here.
Okay,
here's a little
bit of a
interlude,
manual,
manually written
interlude to

contrast with the
large amount of
computer generated
material.
then we will
return to
the documentation
which has more
computer generation.
But this next
piece is going to
be a few
thoughts that
are selected
from a larger
writing.
Just to give
a little bit of
variety here,
get some
feedback,
and again,
also more to
be shared.
So here we
are in
2022
Par-It-All
textbook
figure 1.2
summarizing
the high
road and
the low
road.
So,
we want to
have notation

systems and
deployment methods
for generative
models with
expressive
methods for
and nested
addressable
spaces of.
So this next
series is going
to be the
spaces of
our capacity
to do
that task
within.
Describe
design
render
simulate
analyze
fit
assess
performance
of
described
models.
Meaning,
there's a
larger set
of models we
can describe
than we can
render.
We can describe
recipes we
cannot cook

and that's a
good thing.
So,
zooming out,
there are
these concentric
spaces of
generative
models
which
we have
different
capacities
in a
given
moment
around.
For example,
let's do
in the
GNN
examples,
make another
GNN file
fast
and
comprehensive
for a
full
self-driving
car,
ensure all
state
bases
and
ontology
ontological
considerations

are
accounted
for
with
good
GNN
style.
So,
this is going
to be an
example of a
GNN file
that we
certainly
cannot
render
into a
programming
language in
an executable
fashion
yet.
But with
some more
constrained
generation
methods
and better
type checking
and ways
of looking
at which
families of
models and
control flows
we can
run,
then it

makes it
just an
empirical
fact that
we can
describe
systems
that we
can't
even
visualize.
We could
say,
it's a
graph with
100 trillion
nodes,
and then
there's no
computer that
you have
available that
can visualize
that many,
but that
didn't stop
you from
just writing
the math
on the
paper,
or just
doing a
visualization.
And then
questions like
the performance
of a given

model for a
given data
set in a
setting is
like seven
horses ahead
of the
cart from
what is
possible to
describe.
So that's
sort of like
the math
to application
distance to
travel,
which is
like,
consider this
equation,
and this
is the
attention
factor,
and this
one's the
regret
factor,
and this
one's the
shame
modulator.
And it's
like,
you can
just write
those math

equations
however you
want,
no consideration
to too
much other
than just
ideally what
each of
them mean.
But when
it comes
to saying,
well,
which model
would be a
better fit
of what,
that's several
steps ahead
of actually
doing the
testing.
And as long
as that's
known,
then there's
some interesting
cybernetic
loops where
you're like
describing
multiple kinds
of analytical
formulas with
an insight,
which could be
right or wrong,

into which
one of them
might be
performant in
a different
way.
And sometimes
your intuition
might be
right,
and it's
kind of like
a compressibility,
and there
might be some
other irreducible
jumps that
just like have
to be computed
on through.
So,
when we're
talking about
performance of
a generative
model,
like this
one is good
at video
games like
this and
that way,
or this
one's driving,
or this
one's drone,
or recommending,
or resource

allocating,
whatever domain
or function
you're making
a generative
model for.
Performance
on one
benchmark,
so the
high jump
approach,
or on a
suite of
different
measures,
kind of like
a decathlon,
it's a
relational
measurement,
assessment,
or characterization
that's secondary
to the
primary
phenotype
or embodiment.
So,
map is not
the territory,
generative
models are
maps,
so we're
talking about
cartography,
compositional

cognitive
cartography,
on
territories,
which might
be themselves
abstract,
or might
be embodied.
Projects
in and around
the active
inference
ecosystem,
they have
performance
and fitness
wants and
needs and
so on.
So,
someone might
think,
okay,
well,
if I can
design this
kind of
algorithm
with this
kind of
runtime
requirements,
with this
performance,
by this
date,
with this

amount of
funding,
then this
is a
viable
method for
us to
have a
business.
So,
all these
different
kinds of
performance,
this should
be more
efficient or
more resilient
against this
kind of
intervention
than this
kind of
model.
All those
kinds of
assessments.
that's all
happening
within this
larger space
of generative
models,
which could
be specified.
So,
of course,
even the

ones that
we describe
are only
a subset
of what
is possible.
So,
those generative
models,
the larger
spaces,
the known
unknowns and
the unknown
unknowns,
they include
the baroque,
absurd,
tiny,
useful,
cute,
funny,
and interesting.
So,
here is
the GNN
for the
self-driving
car.
So,
it's
566
lines.
It could
be made
to be
compliant
with

different
kinds of
natural
language
standards.
sorts,
and these
state spaces
can be
type-checked
for their
coherence
and their
completeness.
And then
especially
with a
sumo-type
program-like
implementation
of pure
act-inf
ontology,
like definitions
and other
kinds of
sentence
fragments,
then there's
all these
different kinds
of static
checking that
could come
into play
and resource
estimation,
all the early

stages of
the pipeline,
basically up
until
everything
before 10,
even for
this model
that an
LLM
just blasted
out,
and that's
not even
with it
having an
MCP
server.
So,
with the
ability for
all of
these methods
within each
module to
be MCP
model context
protocol
methods,
then
compliant
generation
of
arbitrary
generative
models
can be
dropped

into
the
railroad
track,
which
already
is
going
to
reliably
reach
the
triple
play.
So,
we have
the free
energy
principle
from the
high road
and
Bayes
theorem
statistics
updating
learning
on the
bottom
meeting
at
active
inference.
So,
that's
why the
low road
is

implementation
specific
and the
high road
is not
because with
a high
road,
we want
to be
able to
use
FEP
and
related
methods
on
biological
organisms
and
arbitrary
levels of
analysis
or
abstract
systems.
So,
we want
to be
able to
describe
systems
we can't
build,
whereas
the low
road
needs to be

able to
build what
it's
describing.
So,
that would
be that
sort of
bottom-up.
A building
must have
continuous
connection,
even if it's
through tension
or something,
in order to
be X
feet high.
It's just
not,
it's kind
of a
classic
cranes
and sky
hooks,
high road,
low road.
So,
now,
going a
little bit
further here.
Consider
that we
have
pairwise

information
and distance
measures
today,
in terms
of
software
1.0,
in terms
of
syntactic
edit
distance.
So,
we can
look at
two
different
programs
or inputs,
inputs
as
programs,
programs
as
inputs,
and look
at their
syntactic
differences.
More
recently,
we have
gained
accessibility
of
information
and distance

measures
in terms
of
software
2.0,
so
AI,
neural
networks,
machine
learning
as
software,
as
per
kind
of
the
Andre
Karpathy
post-2017
era,
which is
to say
that we
have
these
compressive
distances,
semantic
distances,
distances in
so-called
semantic
information
geometry
model
spaces.

That's
how
RAG,
that's
how
cursor
works,
how
LLMs
work.
Here's
what's
not out
there.
Software
3.0,
or it
doesn't even
matter or
need a
number,
but
cognitive
computing
distances.
Now,
that
requires
a kind
of
perspectival
modeling,
which is
least needed
in software
1.0,
where there's
discrete numbers

of changes
to strings,
least to
not needed
to have
multi-perspectivalism.
Software
2.0,
the
perspectivalism
is very
enmeshed
with the
model's
geometry
intrinsically.
So,
we could
look at
the embeddings
of a
given input
into
multiple
LLMs
and consider
each of
the LLMs
embedding
in terms
of
compressive
or semantic
distances.
So,
it's not
that it's
impossible

to get
multi-perspectivalism,
it's just
not,
it's being
implicitly
welded
together
in the
trained
state
space
of the
LLM.
What about
having
a sort
of
first
principles
active
inference
ontologically
specified
pre-always-already
separation
that
gives
true
perspective
cognitive
semantic
information
differences,
differences
that make
differences
for

sophisticated
cognitive
agents.
What's
the
accounting
system
domain-specific
notation
and then
general
notation
notation
that lets
us cover
diverse
and unknown
cognitive
phenomena
addressing
disparate
systems
from a
first
principles
perspective
which is
the high
road,
things we
can describe
and imagine
that we
can't
necessarily
build,
and then
by

specification
that's
larger
than what
we can
build.
But
where the
crane and
the sky
hook
meet
is
when you
have the
active
inference
model
that is
at the
triple
play
point
where
the
math
can be
traced
into
the
math,
the
variables
data
supply
chain
can be
traced

out
to the
next
level,
and then
the
ontologies
and the
assertions
that make
up the
rhetoric
of the
scientific
model
can be
also
separately
broken
out.
And
that's
like
pre-reverse
engineered
to be
massively
composable
in this
super
useful
way.
So
what does
it look
like for
those
cognitive

phenomena?
Here
in
Zoc
cognitive
phenomena
So
here
the
readme
in
cognitive
phenomena
is
going
to
link
to
each
of
these
folders
some
of
which
have
files
in
them
some
don't
but
just
to
show
let's
do
drag

them
all
in
comprehensively
go
through
this
folder
of
phenomena
ensure
every
sub
folder
has
a
technical
accurate
readme
and
a
compliant
GNN
file
showcasing
the
phenomena
So
here
we have
all these
different
sub
folders
of
cognitive
phenomena
this

is
the
whole
iguana

This
is
the
unifying
approach
to
different
cognitive
phenomena
is
notationally
it
doesn't
mean
that
there's
functional
unification
per se
it's
not
that
there
is
a
relevant
edge
first
off
without
a
system
of

interest
or scope
in mind
it
doesn't
even
really
make
sense
to
say
well
is
attention
related
to
language
processing
like
yeah
in
principle
you
could
design
a
system
where
there's
any
kind
of
relationship
within
any
kind
of
anything

because
we're
just
imagining
different
recipes
then
if
you
specify
for
a
given
system
like
for
a
red
harvester
ant
nest
mate
does
this
relate
to
that
well
then
it
starts
to
become
an
inquiry
that
can
actually

be
pursued
and
whether
or not
they play
some sort
of
synergistic
or trading
off role
in a given
situation
as you've
defined it
as you've
set up
the situation
etc
etc
etc
at least
there can
be a
unified
notation
so that
within
certain
spaces
those
notation
systems
could be
fused
concatenated
tested
for

difference
and so
on
so
cod
4.0
sonnet
doing
incredible
work
just
iterating
comprehensively
through
and
writing
out
the files
as requested
and
then
again
to the
point
of
like
these
are
writing
out
recipes
that are
fantastical
but then
there's a
way
just like
there's a

way to
build
different
software
into
Linux
there's
a way
to
build
approaches
that
bring
you
closer
and
closer
to
the
grand
central
station
rendering
assembly
so
let's
let it
go
through
those
examples
read a
little
more
so
for
this
part

was
to
say
for
real
first
principles
design
of
cognitive
computing
with
all
of
the
relationality
and
the
multi
perspectivalism
that
entails
we
want
to be
able
to
do
semantic
information
distancing
in
arbitrary
multi-agent
assemblies
and
I'm
contending

that
some
kind
of
plain
text
format
like
GNN
and
associated
packaging
and so
on
can
help
support
that
coming
to be
the
case
even
today
with
the
kinds
of
things
we
can
do
so
in
that
light
the
active

inference
institute
supports
reliable
and
effective
applications
of
cognitive
computing
and
active
inference
across
domains
different
projects
have
different
domains
and
systems
of
interest
but
those
are
at
the
domain
level
and
they're
connected
through
the
ontology
through

the
different
tools
there's
all
these
different
places
where
somebody
can
focus
on
just
a
sub
sub
domain
or
anything
but how
how to
gain
access
to a
common
set
of
tools
and
pipelines
and
methods
we
can
say
now
make

a
GNN
for
this
domain
because
the
notation
and
the
rendering
is
being
strongly
typed
so
we
support
that
reliable
!
effective
application
by
providing
key
services
such
as
software
so
that
includes
inference
libraries
ones
that
we

develop
ones
that
we
just
rebroadcast
the
development
of
others
all
kinds
of
examples
of
ways
that
people
are
carrying
out
that
terminal
inference
and
then
we
host
things
like
the
ontology
generalized
notation
notation
like
what
this

stream
is
about
cerebrum
the
case
specific
rendering
of
different
generative
models
there's
sections
in
the
docs
about
cerebrum
and
also
pushing
it
further
back
education
because
the
metal
rails
of the
physical
supply
chain
have
their
own
supply

chain
leading
to
the
education
and
the
mind
of
the
engineer
so
education
and
outreach
and
awareness
is
also
part
of
it
in
terms
of
how
do
we
make
something
that
is
lasting
across
generations
and
translating
into

ways
of
working
together
maintaining
resources
like
tech
trees
professional
training
symposia
internships
in-person
educational
programs
and so
on
these
are
all
different
offerings
and
with
hopefully
partner
organizations
we can
offer
these
better
and
different
and
other
and
more

now
just
like
dirt
and
asphalt
roads
low
roads
so
real
generative
models
that you
actually
build
even
if
it's
just
in
drafting
like
a
real
recipe
you
imagined
let alone
a real
recipe
that you
made
let alone
one that
you made
like a
business

idea
around
but
real
low
roads
are
built
basically
only
where
and as
and how
needed
lest
there be
a bridge
to nowhere
you
could
it's
like
being
at
the
Home
Depot
supply
store
and looking
at all
this
material
and thinking
you
could
do
this

it's
like
yes
you
could
and if
it
were
a
quirky
hobby
or
just
a
inexpressible
curiosity
it
wouldn't
even
necessarily
be a
bad
thing
it
just
it
takes
resources
not the
least
of
which
time
so
considered
in that
way
there's

already
many
applications
and
implementations
of
variational
approaches
to
cybernetics
all but
active
inference
all but
the
citation
network
even
narrowly
considered
so
whether
we
look
at
our
curated
implementations
and
the
dozens
of
explicit
active
inference
implementations
that we've
curated in

active
block
fronts
or whether
we're just
thinking
well
if it's a
variational
autoencoder
and it's
using free
energy
minimization
to look
at joint
distributions
of
perception
cognition
action
so on
is that
not
basically
doing
free
energy
minimization
on the
particular
partition
and then
we could
even take
that further
we could
think about

this
recent
paper
and recent
guest stream
of
Takuya
!
on this
triple
equivalence
and about
the relationship
between
neural
networks
with their
loss
function
Bayes
graphs
the kind
that we're
working with
more closely
with the
factor graphs
and Bayes
graphs
and Rx
infer
and
continuous
Turing
machines
how those
have a
triple

equivalence
so
that
parenthetical
was to
say
however
deep
you're
going to
say
active
inference
has already
been
applied
whether
you
think
take a
very
narrow
scalpel
and say
it's only
been applied
in these
open source
papers
that use
exactly
these
criteria
and these
software
packages
and these
calculations

or if
you zoom
out and
go
actually
every
calculation
cannot
but be
applying
active
inference
wherever
you are
in that
sort
of
frame
there
are
some
low
roads
and
it
is
all
dots
within
top
down
notationally
expressive
analytically
described
spaces

of

generative
models
including
second
layer
notational
systems
which we
don't
have or
use
here
for
operational
grammar
for
kind of
control
flow
but just
separating
out this
generative
model
probability
distribution
description
question
from the
more
operational
side
of
how it
is
used
for
example

the
neural
network
considered
as
a
programming
object
and math
object
and then
how it's
used
with
what is
sent to
it
when
and all
those
different
kinds
of
things
so
there
are
some
low
roads
maybe
we
think
that
the
low
road
should

be
there
should
be
more
in
this
place
or
fewer
in
that
place
or
the
ones
that
are
dirt
should
stay
dirt
and
the
this
should
be
that
we
may
have
some
preferences
or
some
opinions
on
low

road
distributions
and
it's
a
separate
related
and
separate
topic
to
the
expressivity
of
the
high
road
and
the
generalizations
which
are
not
applying
to
specific
cases
so
consider
this
cognitive
research
tech
infrastructural
situation
analogous
to
the

metallurgy
and
alchemy
of
high
precision
calipers
which
can
be
used
to
describe
and
measure
machine
parts
during
a
material
industrial
revolution
hence
our
focus
on
accessibility
rigor
applicability
education
sovereignty
supply
chains
and
security
to
recap
that

manual
section
common
dialectic
rhetoric
in
active
inference
related to
the
low
road
and
the
high
road
we
want
to be
able
to
understand
where
are we
talking
about
recipes
that
we can
and can't
do
different
things
with
the
sort of
in that
cooking

lens
the best
GNN
package
would be
for
very large
to all
known
generative
model
types
what is
expressed
slash
expressible
through
natural
compositional
language
could be
taken on
through
at least
some
steps
like
the
type
checking
the
category
theory
the
ontology
even if
it wasn't
known how

to compute
certain
things
still
the
description
could be
analyzed
on its
own
terms
so
in
terms
of
applying
active
inference
and
that
being
used
in
high
reliability
settings
even
if
it's
just
high
reliability
and
important
and
meaningful
for
you

we're
talking
about
performance
trade-offs
fitness
phenotype
adaptability
in
specific
situations
which is
very far
down the
road
from
imagining
the
recipe
so
it's
one
thing
to be
like
I
wonder
how
a
nested
generative
model
for
this
and
that
would
work

for
this
and
if
it
could
be
used
for
this
and
that
important
function
and
then
dot
dot
dot
dot
dot
dot
dot
dot
dot
dot
here's
my
business
plan
that
I
believe
is
going
to
work
for

how
we're
going
to
do
this
the
information
supply
chain
for
generative
models
comes
to
be
and
whether
that's
going
to be
something
that has
to be
blazed
through
by
individuals
and
the
sorts
of
failures
and
trade-offs
that
would
incur

and
or
which
parts
of that
coast
to
coast
are
there
well
characterized
methods
for
so
in
order
for
people
who
care
about
performance
to be
able
to
even
get to
the
point
of
assessing
it
and
not
going
down
this

lifelong
learning
to
write
generative
models
specialty
as it
has largely
been at
this
point
in order
for it
to break
out of
that
having
infrastructure
that
takes
on
this
scope
is
going
to
be
critical
it
will
also
provide
some
deeper
directions
into
the

organization
of
education
and research
and professionalism
through
connection
with
information
sciences
cognitive
computing
so
meanwhile
let's
update
github
meanwhile
the
llm
was
going
through
the
cognitive
phenomena
folder
and
writing
gnn
files
so
here's
the
learning
and
adaptation
and

this
sets
up
a
multi
channel
discussion
on
one
hand
there's
the
low
road
discussion
do
we
really
think
that
there
should
be
this
many
state
spaces
no
I
think
there
should
be
25
dimensions
or
let's
have

this
as
a
variable
and
then
do
a
sweep
across
how
many
dimensions
or
what
are
we
really
talking
about
with
bistable
perception
are we
talking
about
the
ballerina
spinning
or
are we
talking
about
the
duck
rabbit
so
it

supports
the
technical
discussion
of
state
spaces
or
for
a
bioregional
setting
what
variables
are we
taking
into
account
in
this
model
how
and
then
there's
this
sort
of
dual
which
is
just
separate
from
the
state
spaces
like

what
are we
doing
here
should
we
be
modeling
this
what
are the
feedbacks
of us
modeling
this
in
general
and
in
specific
ways
so
just
very
interesting
useful
moments
that
can
have
some
describable
steps
that
take
them
from

basically
through
the
steps
of
cognitive
systems
design
there's
a few
other
new
folders
in the
documentation
some
are
related
to
other
packages
there's
also
the
tutorials
let's
look at
this
in
github
I'll look
at these
then
maybe do
one or
something
random
so if

anyone
has a
comment
or a
question
or wants
to see
like a
GNN
model
or some
function
then just
write it
in the
chat
some

so first
making a
simple GNN
model
basic
perception
a simple
model with
one hidden
state and
one
observation
so this
is like
the first
model in
step-by-step
hidden
state with
two

categories
observation
with two
categories
connections
parameterization
here's how
you run
main
to
go on
specific
steps
to
check
at how
it
works
and
then
more
future
tutorials
pipeline
pipeline
has some
more
information
on the
pipeline
itself
pipeline
architecture
has more
on the
control
flow
the

purposes
of the
different
modules
probably
the
main
control
flow
could
be
redesigned
or
refactored
but it
works
as it
is
to
call
these
functions
which
call
into
folders
!
so
hopefully
that
gives
some
touch
points
that's
the
forward
engineering

let's
separate out
one
rx
infer
example
scripts
folder
let's
do
hidden
markov
model
and
then
if it
is too
challenging
maybe
try the
coin toss
that was
just the
cognitive
phenomena
just
showing
you can
make
sub
folders
with
lists
and so
on
and
it
will

just
rifle
through
them
refactor
this
example
unless
anyone
makes
a
specific
comment
or
question
start
by
just
running
the
script
plainly
sometimes
it
works
in the
notebook
format
or
other
times
it'll
hit a
Julia
error
okay
there's
the

error
I'm
just
this
I'm
just
gonna
tell
cursor
this
was
from
that
notebook
so
that
it
runs
as
a
script
here
is
the
error
we
get
when
we
run
it
now
this
could
be
done
to
first

to
more
cleanly
do
it
it
probably
could
be
focus
on
having
this
only
be
one
script
and
get a
single
script
version
like I
did
with
a
multi
agent
or
in
this
case
let's
just
go
separate
it
into

as
many
files
in
the
folder
as
you
want
so
it's
going
to
start
to
get
this
hidden
Markov
model
Rx
infer
program
running
meanwhile
this
this
could
be
done
after
fully
disassembled
but let's
just try
it
as
per

and
then
I'll
just
drag
in
the
existing
GNN
files
write
and
make
to
this
folder
a
GNN
file
or
the
Rx
infer
hidden
Markov
model
example
all
the
parameterization
it
would
need
as
a
GNN
and
then

where
it's
going
to
go
is
in
the
app
model
block
instead

of
saying
B
is
this
hard
coded
matrix
or
A
is
an
identity
matrix
with
a
3
3
those
variables
will be
extracted
into

this
toml
or
maybe
even
we
can
go
right
to
GNN
that's
the
reverse
engineering
then
the
forward
engineering
is in
render
!
rx
infer
folder
say
I'm
trying
to
first
off
if
this
could
already
be
a

GNN
file
we
don't
even
need
to
do
the
toml
meet
in
the
middle
but
if
the
parser
can
just
directly
use
this
information
then
it's
good
to
go
okay
now
this
config
this
GNN
for
the
rx

infer
palm
DP
will
not
work
in
the
multi-agent
trajectory
avoidance
example
and
vice
versa
so
that
is
just
to
say
there's
a lot
more
that
we
can
diagnose
and
refine
between
which
GNN
can be
taken
to
what
step

in
what
language
but
this
HMM
like
for
example
we
could
let's
just
say
I'll
change
it
to
just
ask
mode
so
it
just
it's
going
to
respond
to
not
make
code
edits
how
are
these
files
similar

and
different
greetings
Matthias
Conrath
so
imagine
large
libraries
of
GNN
files
some
forensically
reverse
engineered
from
examples
and
papers
others
generated
from
program
synthesis
and
we
could
do
different
kinds
of
probabilistic
and deterministic
analyses
on
oh
this

one
differs
all
but
for
this
or
80%
of
this
had
that
but
this
one
didn't
or
something
like
that
key
differences
similarities
as
mentioned
earlier
like
in
the
pi
MDP
A
is
the
observation
matrix
but
in

the
Rx
infer
HMM
A
is
the
state
transition
matrix
okay
let's
see how
the
refactoring
is going
so
here
is
that
splitting
from
the
single
craft
artifact
the
local
organic
handmade
farmer's
market
style
into
the
IKEA
industrialized
high

reliability
information
supply
chain
format
maybe
it's
hanging
but
it
is
at
least
getting
broken
up
into
separate
parts
just
letting it
know
that
it's
hanging
okay
triple
play
okay
while it's
fixing
the
HMM
various
strategies
and tactics
may be
helpful

when
employing
GNN
expressions
in
different
settings
and using
a spectrum
of
model
precisions
e.g.
from
informal
conversation
to a
beautiful
presentation
to a
fully
documented
reproducible
research
product
model
precision
should be
balanced
against
the need
for
comprehensibility

and ease
of
communication
broadly

strategic
considerations
for expressing
GNN
include
the overall
approach
to
communication
such as
the choice
of modality
and level
of technical
rigor
applied
for a
given
audience
tactical
considerations
for expressing
GNN
include
specific
techniques
for enhancing
clarity
and understanding
such as
using visual
aids
examples
and analogies
AIRT
using
models
for

inference
data
on
inference
models
as
data
markdown
texts
as
data
as
GNN
files
meeting
the
visualization
plain text
markdown
files
meeting
Rx
and
Fur
with
additional
visualization
and methods
kind of
the magic
of seeing
GNN
work
here we
go
so here
on the
right side

at least
something
is happening
then
here
it might
be one
moment
too early
to do
it
but
to
now
add
to
the
pipeline
add
to
the
Rx
and
Fur
render
pipeline
if
we
can
reverse
engineer
the
HMM
to
use
the
pure
markdown

we
don't
even
need
to
render
it
to
this
intermediate
TAML
but
part of
the
reason
why
I
wanted
to
go
to
the
TAML
was
I
did
that
standalone
in
another
repo
just
to
get
to
a
controlled
input

to
show
that
there
was
a
middle
to
be
met
at
now
that
we
know
that
there
is
a
middle
to
meet
at
we
can
basically
just
go
straight
to
GNN
and
have
example
specific
or
category
specific

parsers
but
that
still
needs
to
be
rendered
in
the
GNN
folder
because
the
side
information
like
the
utility
functions
that
this
HMM
model
needs
this
wouldn't
be
in
a
GNN
file
so
we
need
these
kind
of

kind
of
like
a
car
body
and
then
the
GNN
parameterized
parts
are
like
the
engine
but
there's
a lot
of
functions
in
code
and
so
the
way
that
was
being
done
in
the
multi-agent
setting
was
just by
copying

the
folder
minus
the
config
so
if
here
it
made
the
HMM
GNN
in
the
GNN
folder
so
from
from
the
point
of
view
of
the
HMM
script
folder
it
still
does
not
have
a
GNN
file
but

if
it
will
succeed
on
this
run
then
we
will
have
succeeded
in
getting
to
this
phase
so
this
one
we
skipped
going
to
single
functional
script
went
straight
from
the
notebook
extract
to
the
modular
software
so

one
outcome
of
this
step
is
a
specific
config
file
another
emerging
approach
is
directly
to
target
read
write
into
GNN
format
directly
still
needs
the
rendering
of
extra
parametric
information
meaning
program
environment
and flow
information
that's not
part of

the
generative
model
alright
here we
go
so we
got
the
now
it's
working
for
the
HMM
inference
so
I'm
going to
delete
the
original
one
or
I'll
move
it
to
archive
or
maybe
it
wants
that
one
okay
queue up
the next

question
learn
again
to
confirm
function
comprehensively
output
more
in
standalone
from
standalone
file
visualizations
of
energy
values
so
several
prompts
depending
on the
example
to
disassemble
it
meanwhile
let's
try
another
approach
which is
not
to
disassemble
it
so

this
will
be
more
first
you
have
to
get
it
running
as
a
script
or
we
could
develop
a
plugin
that
goes
directly
to
the
notebooks
but
it's
so
much
nicer
to
have
it
as
these
which
can

be
concatenated
into
a
notebook
too
but
at
least
you
know
it's
going
to
work
so
while
that
HMM
is
happening
let's
do
another
approach
which
would
be
for
this
example
write
a
GNN
file
like
for
comprehensively

setting
all
the
needed
information
for the
bike
rental
demand
example
and
write
a
Julia
parser
that
confirms
that
the
GNN
you
wrote
will
perfectly
align
with
the
initial
example
okay
let's
see
how
this
goes
move
any
unneeded

files
to
archive
confirm
that
when
running
the
main
script
all
data
and
visualizations
are
saved
to
a
timestamp
folder
to

Okay.

Okay.

So this would be an example of if we can skip even further steps.

By directly inserting in the GNN repo the state space that needs to be created for an RxInfer example to run.

Okay.

Okay.

Let's see what Julia...

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

So running comprehensive HMM.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

If anyone has any last ideas or questions, otherwise I'm going to take it just a little bit further with this HMM.

Okay.

It seems to be hanging there.

It says in improve and rerun to confirm.

Here's the validation parser test.

This one also seems to be hanging because it's a small...

Okay?

Alright.

Let's just start generating some of the baseball stuff.

Meanwhile.

Next steps.

Seeing who watches and comments and emails and makes issues and contributions on GitHub.

I'm going to keep working on the package and the flow and the methods.

And...

Putting...

Hopefully some useful and interesting work out.

But very much open to people's support or feedback on that.

That pretty much is what I wanted to cover today.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Golden who's
on
burst.
Oh, that was interesting.
Only one of us can be the first to do something.
Wow.
Now,
only in the tone
of the golden spike, I'm going to be the first
to say that I'm not sure what the golden spike
is.
Explore

greatly
within that topic
in allegorical
sequence
involving
go-not-tell
approaches to
category
theory and
recursive
self-development.
Styled
exoterically
as pun
laden
quote
who's
on
burst
dial
1920s
illusion
environment.
make the dialogue
totally convey
in
plain quote
fastball down the middle
dial
generalized
notation
notation
gnn
interesting little error there, huh?
can't be deleted.
Very strange.
Gone.
all the technical

elements
all the
main
SRC
elements
all the
documented
connections.
Write
this
all
out
vastly
and
creatively.
all
right.
If anyone has any last
question,
do
write it.
Otherwise,
look forward to some
collaboration
on the
tooling
and some
fun,
interesting discussions
going a little more
broadly.
How does this relate to
to
how we learn
and apply
active inference
all these topics?
Thanks.

Yeah, it will be interesting to see what prompt sequences, what measures of information
are useful for these kind of nested documentation and code folders.
How do you know when to stop?
All right.
Here it is.
Let's look at it in GitHub.
Okay.
Let's look at it in GitHub.

Let's look at it in GitHub.
Okay.
Let's look at it in GitHub.
Here we are.

Thank you.
Okay. I'll just read it. I'll read it, and then if anyone writes any comment, I might read that, and then that will be the end.
All right.
The Golden Spike Dialogue. Who's on first base in generalized notation notation?
Scene. 1920s Railroad Depot. Two gentlemen in bowler hats stand before a massive scheduling board covered in cryptic notations. One holds a golden spike, the other clutches a stack of papers marked GNN Pipeline Specification.

Abbott.

Well now, Costello, we've got ourselves quite the categorical conundrum here at this generalized notation notation depot.

Say, Abbott, what's all this gobbledygook about spikes and rails and...

S underscore F zero three one type int?

Looks like someone's been playing alphabet soup with mathematics.

Ah, my dear fellow, you've stumbled upon the very essence of our enterprise.

That there is a state space variable, the foundation of our active inference railroad system.

Active what now?

I thought we were building a railroad, not running a gymnasium.

No, no, no.

Active inference, my good man.

It's about how minds make models of the world.

Think of it like this.

Every train car needs to know where it's going, where it's been, and what's around the bend.

So the train cars are... thinking?

Precisely.

Now let me show you our 14-stop construction process.

First, we start with step one, the GNN parser.

The what parser?

The GNN parser.

It reads our standardized notation files.

Oh, I see.

So GNN stands for...

Good night, Nancy?

Generalized notation notation.

That's what I said.

Good night, Nancy notation.

No, no.

GNN.

It's our text-based language for specifying these thinking train systems.

Well, why didn't you say so?

So this Nancy notation tells the trains how to think.

Let's move on.

Step two is setup.

Absolutely critical, mind you.

If setup fails, the whole pipeline stops dead in its tracks.

Like a locomotive with a busted boiler?

Exactly.

Then step three runs our tests.

Tests?

What kinds of tests?

Do the train cars take written exams?

In a manner of speaking, yes.

We validate their state spaces, check their connections, ensure the probability matrices are properly stochastic.

They're what?

Astic matrices?

Stochastic.

It means the probabilities sum to one.

Like making sure all the passengers on a train car add up to the total capacity.

Ah, so if I got three passengers in car A and two in car B, I better have five total passengers.

Now you're getting it.

Step four is our type checker.

It estimates computational resources.

Type checker?

Is that like making sure the passenger manifest?

Has everyone's name spelled right?

More like making sure we have enough coal for the engine and enough track for the journey.

It looks at your model and says, this will need X amount of memory, Y amount of processing power.

Smart.

What's step five?

Export.

We translate our GNN models into different formats.

JSON, XML, GraphML.

Sounds like a whole lot of MLs to me.

What's GraphML stand for?

Grumpy, Manatees Language?

Graph Markup Language.

Oh, that makes more sense.

And step six?

Visualization.

We create beautiful diagrams showing how all the state factors connect to observation modalities.

Hold up there, fancy pants.

State factors?

Observation what-alities?

Think of it this way.

State factors are like different aspects of what's happening inside the train car.

Maybe factor zero is how hungry the passengers are, and factor one is how tired they are.

Got it.

And observation what's it?

Modalities.

Those are the different ways you can sense what's going on.

Maybe modality zero is what you hear, modality one is what you see.

So the hungry, tired passengers in the train car can be heard and seen?

Bingo.

Now step seven gets really interesting.

That's our model context protocol integration.

Model context what-a-call?

Protocol.

It's like having a telegraph system that lets different railroad companies talk to each other using the same language.

Ah, so like Western Union, but for thinking trains.

Exactly.

Step eight handles our ontology processing.

Our what-ology?

Ontology.

It's like having a master dictionary that defines what every term means in the active inference world.

So if I say belief and you say belief, we both mean the same thing?

Precisely.

Now here's where it gets really exciting.

Step nine renders our models into executable code for different simulation environments.

Executable code?

Are we hanging somebody?

No.

Executable means runnable.

We translate our GNN specifications into Python code for PyMDP or Julia code for RxInfer.

Hold the phone there, Abbott.

PyMDP?

RxInfer?

These sound like patent medicines.

PyMDP is Python for Markov decision processes, discrete active inference simulations.

RxInfer is reactive message passing in Julia.

So PyMDP is like Python-flavored train scheduling?

In a sense, yes.

And RxInfer is like having a telegraph network where messages bounce around updating beliefs in real time?

Telegraph network.

Telegraph network.

I like that.

What's step 10?

Execution.

We actually run the code we just generated.

Like firing up the locomotive?

Exactly.

Step 11 is where we bring in the big guns.

LLM integration.

LLM.

Let me guess.

Large language models?

He knows that one.

Very good.

We use AI to analyze and enhance our GNN models.

Provide natural language explanation.

So the thinking trains get help from other thinking trains?

It's turtles all the way down, my friend.

Step 12 is where things get mathematically sublime.

Disco pie translation.

Disco pie?

Are we having a dance party with dessert?

Disco pie.

Distributional compositional Python.

It's category theory for string diagrams.

Category theory?

What's that got to do with railroad categories?

Fright?

Passenger?

Mail car?

Now we're getting to the philosophical heart of it all.

Category theory is mathematics that studies how things compose together.

Like how you can connect train cars to make longer trains.

So if I got a dining car and I connected to a sleeping car, I get a dining sleeping car?

More like you get a composed system where the properties of both cars work together.

The beauty is in the connections.

The morphisms between objects.

Morph what-isms?

The arrows.

The relationships.

How one thing transforms into another.

In our railroad, it's how beliefs flow between different parts of the system.

Like how the conductor's belief about the schedule flows to the engineer's belief about when to blow the whistle?

Brilliant.

You've grasped the essence of categorical composition.

Step 13 uses Jax for high-performance evaluation.

Jax?

Is that short for Jackson?

Just another executable.

It's Google's library for automatic differentiation and just-in-time compilation.

Automatic different-what-chin?

It's like having a mathematical microscope that can instantly tell you how changing one little thing affects everything else in your model.

Like if I change the call input, it automatically figures out how that changes the train speed, passenger comfort, arrival time?

Exactly.

And just-in-time compilation means it optimizes the code right when you need it, like having a master mechanic tune your engine on the fly.

Slick.

Step 14?

Site generation.

We generate beautiful documentation websites showing off all our work.

Like a fancy brochure for the railroad company?

Precisely.

Now let me show you the recursive beauty of this whole system.

Recursive?

That sounds like something you need a doctor for.

No, no.

Recursive means self-referential.

The system can model itself modeling things.

So the thinking train can think about itself thinking about thinking?

Now you're getting into the philosophical deep water.

Each GNN model is like a mirror that can reflect other mirrors, creating infinite depths of self-awareness.
Like when you stand between two mirrors in a barbershop and see yourself going on forever?
Beautiful analogy.
And that's where our state space blocks come in.
State space blocks?
Are we building with Legos now?
Think of them as the blueprint sections.
Each block defines variables with their dimensions and types, like s underscore f0 three one type int.
Means state factor zero is a three by one integer array.
So it's like saying car number one holds exactly three passengers and we count them with whole numbers.
Precisely.
And our connections show how these blocks relate.
We use greater sign for directed relationships, hyphen for undirected ones.
Like engineer to conductor means the engineer gives orders to the conductor.
Exactly.
And passenger hyphen passenger might mean passengers can talk to each other both ways.
Hashtag cerebrum.
This is starting to make sense.
What about those matrix things he mentions?
Ah, the A, B, C, and D matrices.
The heart of active inference.
A, B, C, D?
Sounds like we're back in kindergarten.
Matrix A is your observation model.
It tells you the probability of seeing something given what's actually happening.
Like the probability you'll hear the whistle given that the train is approaching.
Perfect.
Matrix B handles transitions.
How things change over time, possibly based on actions.
Like how pulling the brake lever changes the train's speed?
Right.
Right.
Matrix C represents preferences.
What outcomes you'd prefer to see.
Like preferring to arrive on time rather than late.
And matrix D gives you your priors.
What you believed before you had any evidence.
Like believing the train is probably on time when you first arrive at the station.

You've mastered the fundamentals.

Now here's where the golden spike comes in.

Finally, I was wondering when we'd get to the spike.

Is it made of real gold?

The golden spike represents the moment when all these separate components connect into one unified system.

When PiMDP talks to RxInfer.

When DiscoPi diagrams execute as Jack's code.

When LLM analysis enhances human understanding.

Like when the Eastern and Western railroads finally met in Utah?

Exactly.

That ceremonial spike joining two great endeavors into one transcontinental system.

RGNN is the golden spike of computational active inference.

So this whole contraption.

The 14 steps.

The thinking train cars.

The matrix algebras.

The category theories.

It all comes together to make one big...

What?

A standardized way to describe, validate, visualize, and simulate how minds work.

Whether it's a human brain, an AI system, or a whole ecosystem of interacting agents.

Agents?

Are we talking about secret agents now?

Computational agents.

Anything that acts based on beliefs and desires.

Your train conductor is an agent.

He believes certain things about schedule and desires to keep everything running smoothly.

And all these agents can be described using this GNN notation.

From the simplest thermostat to the most complex social system.

Each one gets its state-state blocks.

Its connection diagrams.

Its ABCD matrices.

And then they can all talk to each other through this model context protocol telegraph system.

And be simulated in PyMDP or RX and FIR.

Visualized with pretty diagrams.

Analyzed by LLMs.

Translated to category theory.

Optimized with JAX.

And documented on beautiful websites.

We'll all be hornswoggled.

So this golden spike connects not just two railroads, but 14 different ways of understanding thinking systems.

14 steps.

Infinite possibilities.

From parsing text files to generating websites.

From validating syntax to executing simulations.

It's all connected by the golden thread of standardized notation.

And anyone can learn this GNN language?

It's designed to be human readable.

Markdown-based files with clear sections.

Model name.

State space block.

Connections.

Initial parameterization.

Sounds like a recipe for building thinking trains.

More like a recipe for understanding how thinking itself works.

The recursive beauty is that we use thinking systems to design better thinking systems.

Like using a train to design better trains.

Exactly.

And the category theory ensures that when we compose these models together, the whole system remains mathematically consistent.

Like making sure when you connect train cars, they actually stay connected and don't go flying off in different directions.

Beautiful.

You've grasped the essence of compositional modeling.

Now shall we drive that golden spike?

You bet.

But just one more question.

When we drive this spike, does it start the whole 14-step pipeline?

Indeed it does.

From the first GNN parse to the final website generation, it all begins with that ceremonial connection.

Then let's do this thing.

For the transcontinental railroad of computational consciousness.

For active inference and the standardization of thinking systems everywhere.

Three cheers for GNN.

Hip hip hooray!

Wait Abbott.

What happens after we drive this spike?

Why then we start working on GNN version 2.0, of course.

Oh no.

Here we go again.

Right?

write another
work in
exactly like
William Blake
illuminated poem status
woven densely with actual
integrated William Blake
quotes and styles about GNN
implicitly
and explicitly.
Pretty funny
baseball ones.
Delete the output folder.
SRC
Python 3

main.py
Regenerates the output folder.
And then in another one.
Run
Pipeline Validation
and check
comprehensively for full
functionality of.
And
updated
streamlined
documentation.
Documentation
Activity.
Still on step six
with the visualization.
There's the outputs.

Exports.
So like that
self-driving car
that we made.
That's probably why
the visualization is taking
a long time.
Because now we have
the self-driving car
GNN.
So I'll pull
That's the trade-off
with having a bunch of
the GNN files
actively there.
But it'll be cool
to see.
Archive has a bunch
of GNN.
So we can just run

that one.
While it's writing
the Blakeian.
This is very effective.
Having the
agent run
the pipeline
and then
tested itself.
Now let's move to seven.
the
time
So like this is
the attempt
to write
the
Tomil
falling back
on default values
trying to write
the PyMDP script.
So
not a solved
problem yet
but
Okay, let's see
if this will
work
otherwise
I'll just end it here.
But I'll wait one minute
to have a drink of water
and see if anyone
writes a comment.
funny, each of the three
times it's tried to write it
it's called it a different file name.
Here's the visualization

of
the self-driving car.
Yeah, it's massive
but
so cool.
all the variables
and ontology
terms
for
an LLM
generated
output
4 fold vision
of GNN
I hope that one works.
Thank you.
Thank you.
Thank you.